

Tut 3:

Lab 1:

	QUERY PLAN text	🔒
1	Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=2.566..6.668 rows=32.00 loops=1)	
2	Buffers: shared read=153	
3	-> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.323..0.326 rows=1.00 loo...	
4	Index Cond: (customer_id = 1)	
5	Index Searches: 1	
6	Buffers: shared read=3	
7	-> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=2.240..6.331 rows=32.00 loops=1)	
8	Filter: (customer_id = 1)	
9	Rows Removed by Filter: 16012	
10	Buffers: shared read=150	
11	Planning:	
12	Buffers: shared hit=148 read=4 dirtied=1	
13	Planning Time: 10.749 ms	
14	Execution Time: 6.742 ms	

Ans:

- Roles Identification:
 - Outer (top) table: The customer table, which is accessed via an Index Scan.
 - Inner (bottom) table: The rental table, which is accessed via a Sequential Scan.
- Why Nested Loop & Hash Join Expense:
 - The customer table returned exactly 1 row (rows=1.00).
 - The optimizer makes a cost-based decision, prioritizing the smallest "effective" table (based on cardinality) to be the outer table.
 - Because the outer table only returned a single row, a Hash Join is deemed "too expensive." Building a hash table in memory requires initial startup overhead that outweighs the cost of simply performing a single direct lookup using a nested loop.
- Efficiency (Inner Table Loops):
 - The loops value on the inner rental table node is exactly 1.
 - In a nested loop, the inner table is scanned N times, where N represents the total number of rows filtered and passed down from the outer table.

- Since the outer table yielded exactly 1 row, the inner loop only needed to execute 1 time.

Lab 2:

	QUERY PLAN	
	text	🔒
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=1.140..1.422 rows=1000.00 loops=1)	
2	Hash Cond: (f.language_id = l.language_id)	
3	Buffers: shared hit=88 read=1	
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.013..0.145 rows=1000.00 loo...	
5	Buffers: shared hit=88	
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=1.112..1.113 rows=6.00 loops=1)	
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
8	Buffers: shared read=1	
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=1.105..1.106 rows=6.00 loo...	
10	Buffers: shared read=1	
11	Planning:	
12	Buffers: shared hit=162 read=2	
13	Planning Time: 2.750 ms	
14	Execution Time: 1.469 ms	

Ans:

- Identify Phases:

- Hashed (Built): The language table. The execution plan shows the Hash node directly above the sequential scan of language, meaning it serves as the inner table scanned to prepare the in-memory Hash Table during the Build phase.
- Probed: The film table. It acts as the outer table, which is scanned once so its join keys can be hashed and checked against the built Hash Table for matches during the Probe phase.

- Memory Check:

- The Hash node (Line 7) displays a Memory Usage of 9kB.

- Hypothesis (SET work_mem = '64kB');

- If you set the working memory to 64kB, the execution will remain exactly the same. Because the hash table only requires 9kB of memory, it easily fits within the new 64kB limit. It will remain an "In-Memory" operation with Batches: 1, staying ultra fast. It will not spill to disk, as Postgres only forces the hash table to split into multiple batches on disk when the required memory exceeds the work_mem threshold.

Lab 3:

	QUERY PLAN	
	text	🔒
1	Merge Join (cost=349.64..547.91 rows=4581 width=23) (actual time=4.507..6.527 rows=4581.00 loops=1)	
2	Merge Cond: (f.film_id = i.film_id)	
3	Buffers: shared hit=88 read=25	
4	-> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.019..0.919 rows=1000.00 loo...	
5	Index Searches: 1	
6	Buffers: shared hit=84 read=1	
7	-> Sort (cost=349.37..360.82 rows=4581 width=6) (actual time=4.470..4.704 rows=4581.00 loops=1)	
8	Sort Key: i.film_id	
9	Sort Method: quicksort Memory: 300kB	
10	Buffers: shared hit=4 read=24	
11	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.073..2.370 rows=4581.00 loops=1)	
12	Buffers: shared hit=1 read=24	
13	Planning:	
14	Buffers: shared hit=43 read=1	
15	Planning Time: 2.454 ms	
16	Execution Time: 7.599 ms	

Ans:

- Structure:

- film f: Uses an Index Scan (film_pkey) to get sorted data "for free" without needing a separate sorting step.
- inventory i: Contains an explicit Sort node (Line 7: quicksort). Because inventory was retrieved using a Sequential Scan (Seq Scan), PostgreSQL had to manually sort its rows on i.film_id before merging.

- Compare (RESET ALL; & remove ORDER BY):

Result: Yes, PostgreSQL switches back to a Hash Join.

- Reason: Without an explicit ORDER BY requirement, PostgreSQL no longer needs the final output to be sorted. Because sorting inventory adds extra CPU overhead ($O(N \log N)$), building a Hash Table and performing a Hash Join ($O(N + M)$) is cheaper and faster for the optimizer than performing an explicit sort for a Merge Join.